大纲：

1、问题回顾

2、技术总结

3、Top方案分享

1、问题回顾

Question: ——

Choices: A—, B—, C—, D—

Student Answer: B—, Correct?

Student Exp: ——

Misconception: —— ← target

2、技术总结

① LLM 结构 Transformer Encoder

如何建模分类任务：

— Last Token Hidden State + Linear → Softmax

— 自回归，约束生成

— 召回式：prefix + candi 1 / prefix + candi 2 / ⋮ / prefix + Candi n } Cross Entropy

② LLM训练

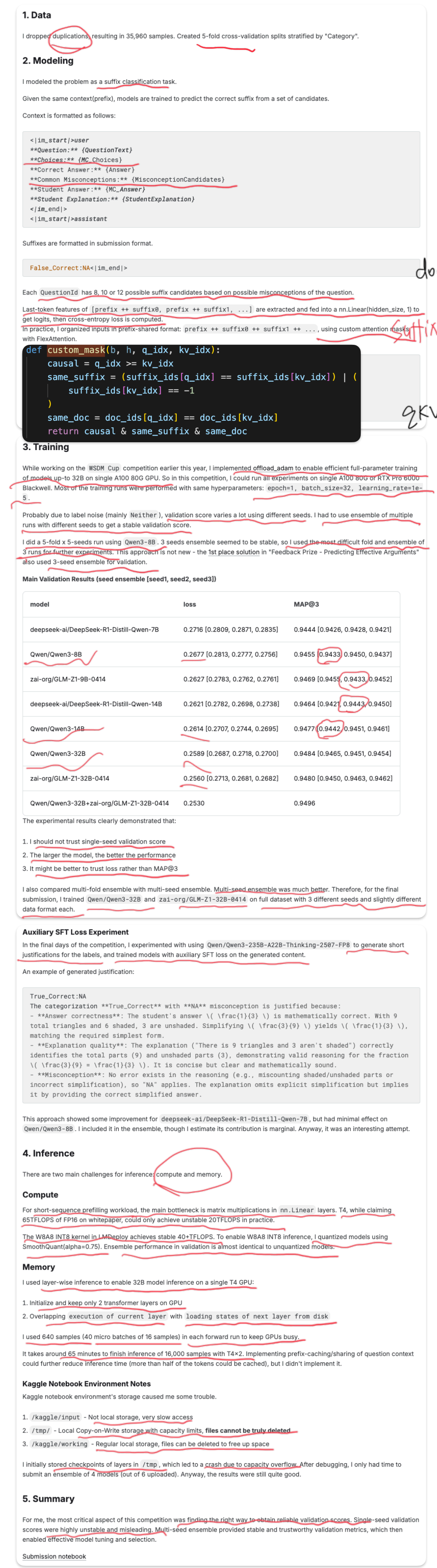
1、LoRA / QLoRA

2、训练-Infra { TP PP DP / 多机多卡

3、工具：{ ms-swift / deepspeed / megatron

③ LLM推理 { Transformers / vllm / 量化

Top1



1. Data

I dropped duplications, resulting in 35,960 samples. Created 5-fold cross-validation splits stratified by "Category".

2. Modeling

I modeled the problem as a suffix classification task.

Given the same context(prefix), models are trained to predict the correct suffix from a set of candidates.

Context is formatted as follows:

```
<|im_start|>user
**Question:** {QuestionText}
**Choices:** {MC_Choices}
**Correct Answer:** {Answer}
**Common Misconceptions:** {MisconceptionCandidates}
**Student Answer:** {MC_Answer}
**Student Explanation:** {StudentExplanation}
<|im_end|>
<|im_start|>assistant
```

Suffixes are formatted in submission format.

```
False_Correct:NA<|im_end|>
```

Each `QuestionId` has 8, 10 or 12 possible suffix candidates based on possible misconceptions of the question.

Last-token features of `[prefix ++ suffix0, prefix ++ suffix1, ...]` are extracted and fed into a nn.Linear(hidden_size, 1) to get logits, then cross-entropy loss is computed.

In practice, I organized inputs in prefix-shared format: `prefix ++ suffix0 ++ suffix1 ++ ...`, using custom attention masks with FlexAttention.

```
def custom_mask(b, h, q_idx, kv_idx):
    causal = q_idx >= kv_idx
    same_suffix = (suffix_ids[q_idx] == suffix_ids[kv_idx]) | (
        suffix_ids[kv_idx] == -1
    )
    same_doc = doc_ids[q_idx] == doc_ids[kv_idx]
    return causal & same_suffix & same_doc
```

doc.ids

suffix_ids -1 -1 -1 0 0 1 1 1

qkv_idx 0 1 2 3 4 5 6 7

Attention 方式：

✱ causal: q_idx ≥ kv_idx

✱ 相同 suffix 可以相互看

suffix 可以看到 doc

✱ 相同doc可以相互看

3. Training

While working on the `WSDM Cup` competition earlier this year, I implemented offload_adam to enable efficient full-parameter training of models up-to 32B on single A100 80G GPU. So in this competition, I could run all experiments on single A100 80G or RTX Pro 6000 Blackwell. Most of the training runs were performed with same hyperparameters: `epoch=1, batch_size=32, learning_rate=1e-5`.

Probably due to label noise (mainly `Neither`), validation score varies a lot using different seeds. I had to use ensemble of multiple runs with different seeds to get a stable validation score.

I did a 5-fold x 5-seeds run using `Qwen3-8B`. 3 seeds ensemble seemed to be stable, so I used the most difficult fold and ensemble of 3 runs for further experiments. This approach is not new - the 1st place solution in "Feedback Prize - Predicting Effective Arguments" also used 3-seed ensemble for validation.

Main Validation Results (seed ensemble [seed1, seed2, seed3])

model	loss	MAP@3
deepseek-ai/DeepSeek-R1-Distill-Qwen-7B	0.2716 [0.2809, 0.2871, 0.2835]	0.9444 [0.9426, 0.9428, 0.9421]
Qwen/Qwen3-8B	0.2677 [0.2813, 0.2777, 0.2756]	0.9455 [0.9433, 0.9450, 0.9437]
zai-org/GLM-Z1-9B-0414	0.2627 [0.2783, 0.2762, 0.2761]	0.9469 [0.9455, 0.9433, 0.9452]
deepseek-ai/DeepSeek-R1-Distill-Qwen-14B	0.2621 [0.2782, 0.2698, 0.2738]	0.9464 [0.9421, 0.9443, 0.9450]
Qwen/Qwen3-14B	0.2614 [0.2707, 0.2744, 0.2695]	0.9477 [0.9442, 0.9451, 0.9461]
Qwen/Qwen3-32B	0.2589 [0.2687, 0.2718, 0.2700]	0.9484 [0.9465, 0.9451, 0.9454]
zai-org/GLM-Z1-32B-0414	0.2560 [0.2713, 0.2681, 0.2682]	0.9480 [0.9450, 0.9463, 0.9462]
Qwen/Qwen3-32B+zai-org/GLM-Z1-32B-0414	0.2530	0.9496

The experimental results clearly demonstrated that:

1. I should not trust single-seed validation score
2. The larger the model, the better the performance
3. It might be better to trust loss rather than MAP@3

I also compared multi-fold ensemble with multi-seed ensemble. Multi-seed ensemble was much better. Therefore, for the final submission, I trained `Qwen/Qwen3-32B` and `zai-org/GLM-Z1-32B-0414` on full dataset with 3 different seeds and slightly different data format each.

Auxiliary SFT Loss Experiment

In the final days of the competition, I experimented with using `Qwen/Qwen3-235B-A22B-Thinking-2507-FP8` to generate short justifications for the labels, and trained models with auxiliary SFT loss on the generated content.

An example of generated justification:

```
True_Correct:NA
The categorization **True_Correct** with **NA** misconception is justified because:
- **Answer correctness**: The student's answer \( \frac{1}{3} \) is mathematically correct. With 9
total triangles and 6 shaded, 3 are unshaded. Simplifying \( \frac{3}{9} \) yields \( \frac{1}{3} \),
matching the required simplest form.
- **Explanation quality**: The explanation ("There is 9 triangles and 3 aren't shaded") correctly
identifies the total parts (9) and unshaded parts (3), demonstrating valid reasoning for the fraction
\( \frac{3}{9} = \frac{1}{3} \). It is concise but clear and mathematically sound.
- **Misconception**: No error exists in the reasoning (e.g., miscounting shaded/unshaded parts or
incorrect simplification), so "NA" applies. The explanation omits explicit simplification but implies
it by providing the correct simplified answer.
```

This approach showed some improvement for `deepseek-ai/DeepSeek-R1-Distill-Qwen-7B`, but had minimal effect on `Qwen/Qwen3-8B`. I included it in the ensemble, though I estimate its contribution is marginal. Anyway, it was an interesting attempt.

4. Inference

There are two main challenges for inference: compute and memory.

Compute

For short-sequence prefilling workload, the main bottleneck is matrix multiplications in `nn.Linear` layers. T4, while claiming 65TFLOPS of FP16 on whitepaper, could only achieve unstable 20TFLOPS in practice.

The W8A8 INT8 kernel in LMDeploy achieves stable 40+TFLOPS. To enable W8A8 INT8 inference, I quantized models using SmoothQuant(alpha=0.75). Ensemble performance in validation is almost identical to unquantized models.

Memory

I used layer-wise inference to enable 32B model inference on a single T4 GPU:

1. Initialize and keep only 2 transformer layers on GPU
2. Overlapping `execution of current layer` with `loading states of next layer from disk`

I used 640 samples (40 micro batches of 16 samples) in each forward run to keep GPUs busy.

It takes around 65 minutes to finish inference of 16,000 samples with T4×2. Implementing prefix-caching/sharing of question context could further reduce inference time (more than half of the tokens could be cached), but I didn't implement it.

Kaggle Notebook Environment Notes

Kaggle notebook environment's storage caused me some trouble.

1. `/kaggle/input` - Not local storage, very slow access
2. `/tmp/` - Local Copy-on-Write storage with capacity limits, **files cannot be truly deleted**
3. `/kaggle/working` - Regular local storage, files can be deleted to free up space

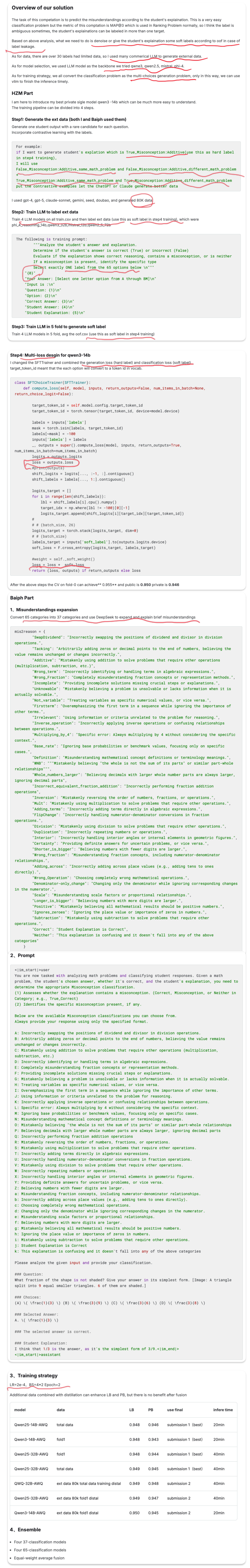
I initially stored checkpoints of layers in `/tmp`, which led to a crash due to capacity overflow. After debugging, I only had time to submit an ensemble of 4 models (out of 6 uploaded). Anyway, the results were still quite good.

5. Summary

For me, the most critical aspect of this competition was finding the right way to obtain reliable validation scores. Single-seed validation scores were highly unstable and misleading. Multi-seed ensemble provided stable and trustworthy validation metrics, which then enabled effective model tuning and selection.

Submission notebook

Top2



Overview of our solution

The task of this competation is to predict the misunderstandings according to the student's explaination. This is a very easy classification problem but the metric of this comptation is MAP@3 which is used in Ranking Problem normally, so I think the label is ambiguous sometimes, the student's explainations can be labeled in more than one target.

Based on above analysis, what we need to do is denoise or give the student's explaination some soft labels according to oof in case of label leakage.

As for data, there are over 30 labels had limited data, so I used many commerical LLM to generate external data.

As for model selection, we used LLM model as the backbone we tried qwne3, qwen2.5, mistral, phi-4.

As for training strategy, we all convert the classification problem as the multi-choices generation problem, only in this way, we can use vllm to finish the inference timely.

HZM Part

I am here to introduce my best private sigle model qwen3 -14b which can be much more easy to understand.
The training pipeline can be divided into 4 steps.

Step1: Generate the ext data (both I and Baiph used them)

Generate one student output with a rare candidate for each question.
Incorporate contrastive learning with the labels.

```
For example:
if I want to generate student's explation which is True_Misconception:Additive(use this as hard label
in step4 training),
I will use
False_Misconception:Additive_same_math_problem and False_Misconception:Additive_different_math_problem
+
True_Misconception:Additive_same_math_problem and True_Misconception:Additive_different_math_problem
put the contrastive examples let the ChatGPT or Claude generate better data
```

I used gpt-4, gpt-5, claude-sonnet, gemini, seed, doubao, and generated 80K data.

Step2: Train LLM to label ext data

Train 4 LLM models on all train.csv and then label ext data (use this as soft label in step4 training), which were phi_4_reasoning_14b,qwen3_32b,mistral_12b,qwen2.5_72b

```
The following is training prompt:
        """Analyze the student's answer and explanation.
        Determine if the student's answer is correct (True) or incorrect (False)
        Evaluate if the explanation shows correct reasoning, contains a misconception, or is neither
        If a misconception is present, identify the specific type
        Select exactly ONE label from the 65 options below \n"""
        "{0}"
        "Your Answer: [Select one letter option from A through BM]\n"
        "Input is :\n"
        "Question: {1}\n"
        "Option: {2}\n"
        "Correct Answer: {3}\n"
        "Student Answer: {4}\n"
        "Student Explanation: {5}\n"
```

Step3: Train LLM in 5 fold to generate soft label

Train 4 LLM models in 5 fold, avg the oof.csv (use this as soft label in step4 training)

Step4: Multi-loss desgin for qwen3-14b

I changed the SFTTrainer and combined the generation loss (hard label) and classification loss (soft label).
target_token_id meant that the each option will convert to a token id in vocab.

```
class SFTChoiceTrainer(SFTTrainer):
    def compute_loss(self, model, inputs, return_outputs=False, num_items_in_batch=None,
return_choice_logit=False):

        target_token_id = self.model.config.target_token_id
        target_token_id = torch.tensor(target_token_id, device=model.device)

        labels = inputs['labels']
        mask = torch.isin(labels, target_token_id)
        labels[~mask] = -100
        inputs['labels'] = labels
        _, outputs = super().compute_loss(model, inputs, return_outputs=True,
num_items_in_batch=num_items_in_batch)
        logits = outputs.logits
        loss = outputs.loss
        #print(outputs)
        shift_logits = logits[..., :-1, :].contiguous()
        shift_labels = labels[..., 1:].contiguous()

        logits_target = []
        for i in range(len(shift_labels)):
            lbl = shift_labels[i].cpu().numpy()
            target_idx = np.where(lbl != -100)[0][-1]
            logits_target.append(shift_logits[i][target_idx][target_token_id])
        #
        # # (batch_size, 26)
        logits_target = torch.stack(logits_target, dim=0)
        # # (batch_size)
        labels_target = inputs['soft_label'].to(outputs.logits.device)
        soft_loss = F.cross_entropy(logits_target, labels_target)

        #weight = self._soft_weight()
        loss = loss +  soft_loss
        return (loss, outputs) if return_outputs else loss
```

After the above steps the CV on fold-0 can achieve** 0.955** and public is **0.950** private is **0.946**

Baiph Part

1、Misunderstandings expansion

Convert 65 categories into 37 categories and use DeepSeek to expand and explain brief misunderstandings

```
mis2reason = {
        "SwapDividend": "Incorrectly swapping the positions of dividend and divisor in division
operations.",
        "Tacking": "Arbitrarily adding zeros or decimal points to the end of numbers, believing the
value remains unchanged or changes incorrectly.",
        "Additive": "Mistakenly using addition to solve problems that require other operations
(multiplication, subtraction, etc.)",
        "Wrong_term": "Incorrectly identifying or handling terms in algebraic expressions.",
        "Wrong_Fraction": "Completely misunderstanding fraction concepts or representation methods.",
        "Incomplete": "Providing incomplete solutions missing crucial steps or explanations.",
        "Unknowable": "Mistakenly believing a problem is unsolvable or lacks information when it is
actually solvable.",
        "Not_variable": "Treating variables as specific numerical values, or vice versa.",
        "Firstterm": "Overemphasizing the first term in a sequence while ignoring the importance of
other terms.",
        "Irrelevant": "Using information or criteria unrelated to the problem for reasoning.",
        "Inverse_operation": "Incorrectly applying inverse operations or confusing relationships
between operations.",
        "Multiplying_by_4": "Specific error: Always multiplying by 4 without considering the specific
context.",
        "Base_rate": "Ignoring base probabilities or benchmark values, focusing only on specific
cases.",
        "Definition": "Misunderstanding mathematical concept definitions or terminology meanings.",
        "WNB": """Mistakenly believing "the whole is not the sum of its parts" or similar part-whole
relationships""",
        "Whole_numbers_larger": "Believing decimals with larger whole number parts are always larger,
ignoring decimal parts",
        "Incorrect_equivalent_fraction_addition": "Incorrectly performing fraction addition
operations",
        "Inversion": "Mistakenly reversing the order of numbers, fractions, or operations.",
        "Mult": "Mistakenly using multiplication to solve problems that require other operations.",
        "Adding_terms": "Incorrectly adding terms directly in algebraic expressions.",
        "FlipChange": "Incorrectly handling numerator-denominator conversions in fraction
operations.",
        "Division": "Mistakenly using division to solve problems that require other operations.",
        "Duplication": "Incorrectly repeating numbers or operations.",
        "Interior": "Incorrectly handling interior angles or internal elements in geometric figures.",
        "Certainty": "Providing definite answers for uncertain problems, or vice versa.",
        "Shorter_is_bigger": "Believing numbers with fewer digits are larger.",
        "Wrong_fraction": "Misunderstanding fraction concepts, including numerator-denominator
relationships.",
        "Adding_across": "Incorrectly adding across place values (e.g., adding tens to ones
directly).",
        "Wrong_Operation": "Choosing completely wrong mathematical operations.",
        "Denominator-only_change": "Changing only the denominator while ignoring corresponding changes
in the numerator.",
        "Scale": "Misunderstanding scale factors or proportional relationships.",
        "Longer_is_bigger": "Believing numbers with more digits are larger.",
        "Positive": "Mistakenly believing all mathematical results should be positive numbers.",
        "Ignores_zeroes": "Ignoring the place value or importance of zeros in numbers.",
        "Subtraction": "Mistakenly using subtraction to solve problems that require other
operations.",
        "Correct": "Student Explanation is Correct",
        "Neither": "This explanation is confusing and it doesn't fall into any of the above
categories"
    }
```

2、Prompt

```
<|im_start|>user
You are now tasked with analyzing math problems and classifying student responses. Given a math
problem, the student's chosen answer, whether it's correct, and the student's explanation, you need to
determine the appropriate Misconception classification.
(1) Assesses whether the explanation contains a misconception. (Correct, Misconception, or Neither in
Category; e.g., True_Correct)
(2) Identifies the specific misconception present, if any.

Below are the available Misconception classifications you can choose from.
Always provide your response using only the specified format.

A: Incorrectly swapping the positions of dividend and divisor in division operations.
B: Arbitrarily adding zeros or decimal points to the end of numbers, believing the value remains
unchanged or changes incorrectly.
C: Mistakenly using addition to solve problems that require other operations (multiplication,
subtraction, etc.)
D: Incorrectly identifying or handling terms in algebraic expressions.
E: Completely misunderstanding fraction concepts or representation methods.
F: Providing incomplete solutions missing crucial steps or explanations.
G: Mistakenly believing a problem is unsolvable or lacks information when it is actually solvable.
H: Treating variables as specific numerical values, or vice versa.
I: Overemphasizing the first term in a sequence while ignoring the importance of other terms.
J: Using information or criteria unrelated to the problem for reasoning.
K: Incorrectly applying inverse operations or confusing relationships between operations.
L: Specific error: Always multiplying by 4 without considering the specific context.
M: Ignoring base probabilities or benchmark values, focusing only on specific cases.
N: Misunderstanding mathematical concept definitions or terminology meanings.
O: Mistakenly believing "the whole is not the sum of its parts" or similar part-whole relationships
P: Believing decimals with larger whole number parts are always larger, ignoring decimal parts
Q: Incorrectly performing fraction addition operations
R: Mistakenly reversing the order of numbers, fractions, or operations.
S: Mistakenly using multiplication to solve problems that require other operations.
T: Incorrectly adding terms directly in algebraic expressions.
U: Incorrectly handling numerator-denominator conversions in fraction operations.
V: Mistakenly using division to solve problems that require other operations.
W: Incorrectly repeating numbers or operations.
X: Incorrectly handling interior angles or internal elements in geometric figures.
Y: Providing definite answers for uncertain problems, or vice versa.
Z: Believing numbers with fewer digits are larger.
a: Misunderstanding fraction concepts, including numerator-denominator relationships.
b: Incorrectly adding across place values (e.g., adding tens to ones directly).
c: Choosing completely wrong mathematical operations.
d: Changing only the denominator while ignoring corresponding changes in the numerator.
e: Misunderstanding scale factors or proportional relationships.
f: Believing numbers with more digits are larger.
g: Mistakenly believing all mathematical results should be positive numbers.
h: Ignoring the place value or importance of zeros in numbers.
i: Mistakenly using subtraction to solve problems that require other operations.
j: Student Explanation is Correct
k: This explanation is confusing and it doesn't fall into any of the above categories

Please analyze the given input and provide your classification.

### Question:
What fraction of the shape is not shaded? Give your answer in its simplest form. [Image: A triangle
split into 9 equal smaller triangles. 6 of them are shaded.]

### Choices:
(A) \( \frac{1}{3} \) (B) \( \frac{3}{9} \) (C) \( \frac{3}{6} \) (D) \( \frac{3}{8} \)

### Selected Answer:
A. \( \frac{1}{3} \)

### The selected answer is correct.

### Student Explanation:
I think that 1/3 is the answer, as it's the simplest form of 3/9.<|im_end|>
<|im_start|>assistant
```

3、Training strategy

LR=2e-4, BS=4*2 Epoch=2

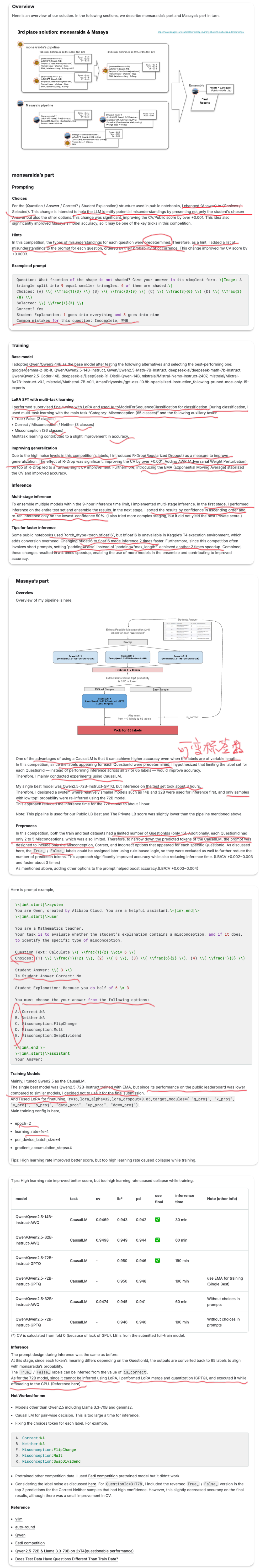
Additional data combined with distillation can enhance LB and PB, but there is no benefit after fusion

model	data	LB	PB	use final	infere time
Qwen25-14B-AWQ	total data	0.948	0.946	submission 1（best）	20min
Qwen3-14B-AWQ	fold1	0.948	0.943	submission 1（best）	20min
Qwen25-32B-AWQ	fold1	0.948	0.944	submission 1（best）	40min
Qwen25-32B-AWQ	total data	0.949	0.945	submission 1（best）	40min
QWQ-32B-AWQ	ext data 80k total data training distal	0.949	0.948	submission 2	40min
Qwen25-32B-AWQ	ext data 80k fold1 distal	0.949	0.947	submission 2	40min
Qwen3-14B-AWQ	ext data 80k fold1 distal	0.950	0.945	submission 2	20min

4、Ensemble

- Four 37-classification models
- Four 65-classification models
- Equal-weight average fusion

Top3



Overview

Here is an overview of our solution. In the following sections, we describe monsaraida's part and Masaya's part in turn.

3rd place solution: monsaraida & Masaya

monsaraida's part

Prompting

Choices

For the (Question / Answer / Correct? / Student Explanation) structure used in public notebooks, I changed (Answer) to (Choices / Selected). This change is intended to help the LLM identify potential misunderstandings by presenting not only the student's chosen Answer but also the other options.This change was significant, improving the CV/Public score by over +0.001. This idea also significantly improved Masaya's model accuracy, so it may be one of the key tricks in this competition.

Hints

In this competition, the types of misunderstandings for each question were predetermined. Therefore, as a hint, I added a list of misunderstandings to the prompt for each question, ordered by their probability of occurrence. This change improved my CV score by +0.0003.

Example of prompt

```
Question: What fraction of the shape is not shaded? Give your answer in its simplest form. \[Image: A
triangle split into 9 equal smaller triangles. 6 of them are shaded.\]
Choices: (A) \\( \\frac{1}{3} \\) (B) \\( \\frac{3}{9} \\) (C) \\( \\frac{3}{6} \\) (D) \\( \\frac{3}
{8} \\)
Selected: \\( \\frac{1}{3} \\)
Correct? Yes
Student Explanation: 1 goes into everything and 3 goes into nine
Common mistakes for this question: Incomplete, WNB
```

Training

Base model

I adopted Qwen/Qwen3-14B as the base model after testing the following alternatives and selecting the best-performing one: google/gemma-2-9b-it, Qwen/Qwen2.5-14B-Instruct, Qwen/Qwen2.5-Math-7B-Instruct, deepseek-ai/deepseek-math-7b-instruct, Qwen/Qwen2.5-Coder-14B, deepseek-ai/DeepSeek-R1-Distill-Qwen-14B, mistralai/Mistral-Nemo-Instruct-2407, mistralai/Mathstral-7B-v0.1, mistralai/Mixtral-8x7B-Instruct-v0.1, AmanPriyanshu/gpt-oss-10.8b-specialized-instruction_following-pruned-moe-only-15-experts

LoRA SFT with multi-task learning

I performed supervised fine-tuning with LoRA and used AutoModelForSequenceClassification for classification. During classification, I used multi-task learning with the main task "Category: Misconception (65 classes)" and the following auxiliary tasks:

- True / False (2 classes)
- Correct / Misconception / Neither (3 classes)
- Misconception (36 classes)

Multitask learning contributed to a slight improvement in accuracy.

Improving generalization

Due to the high noise levels in this competition's labels, I introduced R-Drop(Regularized Dropout) as a measure to improve generalization. The effect of R-Drop was significant, improving the CV by over +0.001. Adding AWP (Adversarial Weight Perturbation) on top of R-Drop led to a further, slight CV improvement. Furthermore, introducing the EMA (Exponential Moving Average) stabilized the CV and improved accuracy.

Inference

Multi-stage inference

To ensemble multiple models within the 9-hour inference time limit, I implemented multi-stage inference. In the first stage, I performed inference on the entire test set and ensemble the results. In the next stage, I sorted the results by confidence in ascending order and re-ran inference only on the lowest-confidence 50%. (I also tried more complex staging, but it did not yield the best Private score.)

Tips for faster inference

Some public notebooks used `torch_dtype=torch.bfloat16`, but bfloat16 is unavailable in Kaggle's T4 execution environment, which adds conversion overhead. Changing bfloat16 to float16 made inference 2 times faster. Furthermore, since this competition often involves short prompts, setting `padding=False` instead of `padding="max_length"` achieved another 2 times speedup. Combined, these changes resulted in a 4 times speedup, enabling the use of more models in the ensemble and contributing to improved accuracy.

Masaya's part

Overview

Overview of my pipeline is here,

Students Answer
Extract Possible Misconception (2~5 labels) for each "QuestionId"
Prompt
CausalLM 1 Qwen/Qwen2.5-32B-Instruct-AWQ
CausalLM 2 Qwen/Qwen2.5-32B-Instruct-AWQ
CausalLM 3 Qwen/Qwen2.5-14B-Instruct-AWQ
Prob for 4~7 labels
Extract items whose top1 probability is 0.95 or lower
Difficult Sample
Easy Sample
CausalLM 4 Qwen/Qwen2.5-72B-Instruct-GPTQ (lora merged)
Alignment from 4~7 labels to 65 labels
is_correct
Prob for 65 labels

可变标签数

One of the advantages of using a CausalLM is that it can achieve higher accuracy even when the labels are of variable length.
In this competition, since the labels appearing for each QuestionId were predetermined, I hypothesized that limiting the label set for each QuestionId — instead of performing inference across all 37 or 65 labels — would improve accuracy.
Therefore, I mainly conducted experiments using CausalLM.

My single best model was Qwen2.5-72B-Instruct-GPTQ, but inference on the test set took about 3 hours.
Therefore, I designed a system where relatively smaller models such as 14B and 32B were used for inference first, and only samples with low top1 probability were re-inferred using the 72B model.
This approach reduced the inference time for the 72B model to about 1 hour.

Note: This pipeline is used for our Public LB Best and The Private LB score was slightly lower than the pipeline mentioned above.

Preprocess

In this competition, both the train and test datasets had a limited number of QuestionIds (only 15). Additionally, each QuestionId had only 2 to 5 Misconceptions, which was also limited. Therefore, to narrow down the predicted tokens of the CausalLM, the prompt was designed to include only the Misconception, Correct, and Incorrect options that appeared for each specific QuestionId. As discussed here, the True_ / False_ labels could be assigned later using rule-based logic, so they were excluded as well to further reduce the number of prediction tokens. This approach significantly improved accuracy while also reducing inference time. (LB/CV +0.002~0.003 and faster about 3 times)
As mentioned above, adding other options to the prompt helped boost accuracy.(LB/CV +0.003~0.004)

Here is prompt example,

```
\<|im\_start|\>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.\<|im\_end|\>
\<|im\_start|\>user

You are a Mathematics teacher.
Your task is to evaluate whether the student's explanation contains a misconception, and if it does,
to identify the specific type of misconception.

Question Text: Calculate \\( \\frac{1}{2} \\div 6 \\)
Choices: (1) \\( \\frac{1}{12} \\), (2) \\( 3 \\), (3) \\( \\frac{6}{2} \\), (4) \\( \\frac{1}{3} \\)

Student Answer: \\( 3 \\)
Is Student Answer Correct: No

Student Explanation: Because you do half of 6 \= 3

You must choose the your answer from the following options:

A. Correct:NA
B. Neither:NA
C. Misconception:FlipChange
D. Misconception:Mult
E. Misconception:SwapDividend

\<|im\_end|\>
\<|im\_start|\>assistant
Your Answer:
```

Training Models

Mainly, I tuned Qwen2.5 as the CausalLM.
The single best model was Qwen2.5-72B-Instruct trained with EMA, but since its performance on the public leaderboard was lower compared to similar models, I decided not to use it for the final submission.
And I used LoRA for finetuning, `r=16,lora_alpha=32,lora_dropout=0.05,target_modules=( "q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj")`.
Main training config is here,

- epoch=2
- learning_rate=1e-4
- per_device_train_batch_size=4
- gradient_accumulation_steps=4

Tips: High learning rate improved better score, but too high learning rate caused collapse while training.

Tips: High learning rate improved better score, but too high learning rate caused collapse while training.

model	task	cv	lb*	pd	use final	inferrence time	Note (other info)
Qwen/Qwen2.5-14B-Instruct-AWQ	CausalLM	0.9469	0.943	0.942	✅	30 min	
Qwen/Qwen2.5-32B-Instruct-AWQ	CausalLM	0.9498	0.949	0.944	✅	60 min	
Qwen/Qwen2.5-72B-Instruct-GPTQ	CausalLM	-	0.950	0.946	✅	190 min	
Qwen/Qwen2.5-72B-Instruct-GPTQ	CausalLM	-	0.950	0.948		190 min	use EMA for training (Single Best)
Qwen/Qwen2.5-32B-Instruct-AWQ	CausalLM	0.9474	0.945	0.941		60 min	Without choices in prompts
Qwen/Qwen2.5-72B-Instruct-GPTQ	CausalLM	-	0.946	0.940		190 min	Without choices in prompts

(*) CV is calculated from fold 0 (because of lack of GPU). LB is from the submitted full-train model.

Inference

The prompt design during inference was the same as before.
At this stage, since each token's meaning differs depending on the QuestionId, the outputs are converted back to 65 labels to align with monsaraida's probability.
The `True_` / `False_` labels can be inferred from the value of `is_correct`.
As for the 72B model, since it cannot be inferred using LoRA, I performed LoRA merge and quantization (GPTQ), and executed it while offloading to the CPU. (Reference here)

Not Worked for me

- Models other than Qwen2.5 including Llama 3.3-70B and gemma2.
- Causal LM for pair-wise decision. This is too large a time for inference.
- Fixing the choices token for each label. For example,

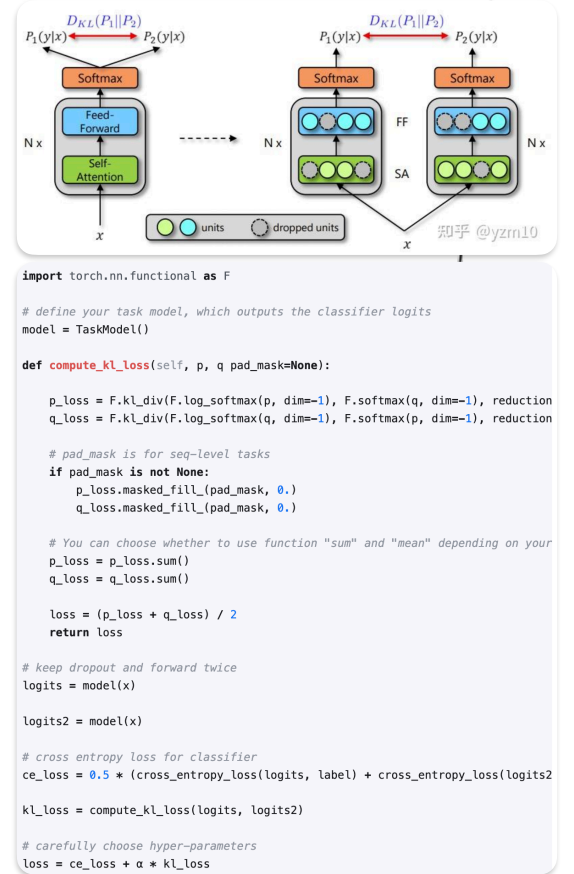
```
A. Correct:NA
B. Neither:NA
F. Misconception:FlipChange
D. Misconception:Mult
R. Misconception:SwapDividend
```

- Pretrained other competition data. I used Eedi competition pretrained model but it didn't work.
- Considering the label noise as discussed here. For `QuestionId=31778`, I included the reversed `True_ / False_` version in the top 2 predictions for the Correct Neither samples that had high confidence. However, this slightly decreased accuracy on the final results, although there was a small improvement in CV.

Reference

- vllm
- auto-round
- Qwen
- Eedi competition
- Qwen2.5-72B & Llama 3.3-70B on 2xT4(questionable performance)
- Does Test Data Have Questions Different Than Train Data?

R-Drop



```
import torch.nn.functional as F

# define your task model, which outputs the classifier logits
model = TaskModel()

def compute_kl_loss(self, p, q pad_mask=None):

    p_loss = F.kl_div(F.log_softmax(p, dim=-1), F.softmax(q, dim=-1), reduction
    q_loss = F.kl_div(F.log_softmax(q, dim=-1), F.softmax(p, dim=-1), reduction

    # pad_mask is for seq-level tasks
    if pad_mask is not None:
        p_loss.masked_fill_(pad_mask, 0.)
        q_loss.masked_fill_(pad_mask, 0.)

    # You can choose whether to use function "sum" and "mean" depending on your
    p_loss = p_loss.sum()
    q_loss = q_loss.sum()

    loss = (p_loss + q_loss) / 2
    return loss

# keep dropout and forward twice
logits = model(x)

logits2 = model(x)

# cross entropy loss for classifier
ce_loss = 0.5 * (cross_entropy_loss(logits, label) + cross_entropy_loss(logits2

kl_loss = compute_kl_loss(logits, logits2)

# carefully choose hyper-parameters
loss = ce_loss + α * kl_loss
```

AWP

AWP（Adversarial Weight Perturbation，对抗性权重扰动）是一种在"权重空间"做对抗训练的技巧。它不是去扰动输入数据，而是对模型参数做一个很小但"最不利"的扰动，让模型在这种不利条件下也能优化好，从而逼迫模型找到更"平坦"的最优点，提升泛化与稳健性。

核心思想与直觉

1 目标是做一个"鲁棒优化"：在每次更新前，先把权重往会让损失增大的方向轻微推一下，再用这个被扰动的权重来计算并更新。等价于在优化时考虑"最坏邻域"的损失，偏好更平坦、泛化更好的解。

2 与输入对抗训练（FGSM/PGD）类似，但扰动对象从"输入"变成了"权重"。输入对抗提高输入层面的鲁棒性，AWP提高参数层面的鲁棒性，减少对尖锐最小值的依赖。

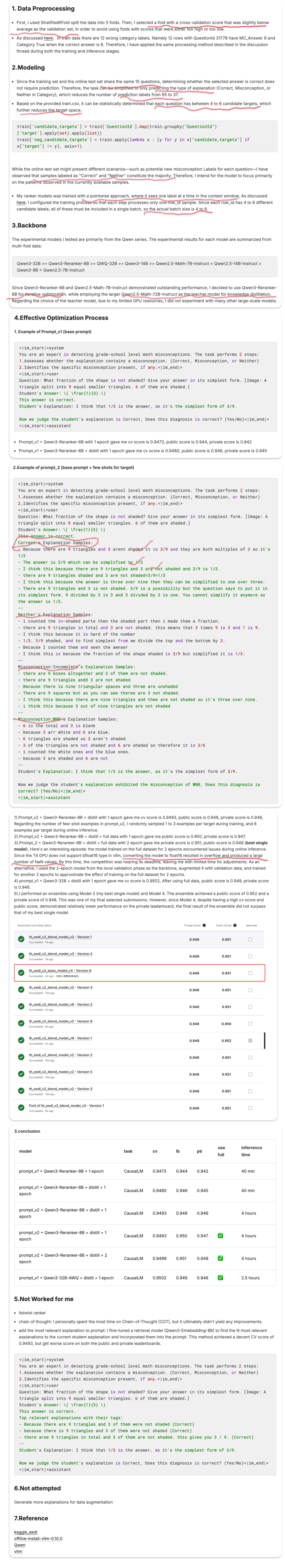
3 在标签噪声较高时，尖锐的极小值往往是在"迎合噪声"的方向；AWP促使模型选择在小范围权重扰动下损失仍不敏感的解，从而更抗噪。

基本算法流程（每个训练批次）

1 备份需要扰动的参数（通常不包含归一化层的缩放/偏置，比如LayerNorm/BatchNorm的gamma、beta，以及bias）。
2 前向-反向一次，拿到对参数的梯度。
3 计算"对抗性权重扰动"：
    1 方向：沿着让损失增大的方向（梯度方向）。
    2 规模：将梯度做范数归一化后乘以一个步长，再对不同参数按其尺度做相对约束，常见做法是限制扰动不超过参数绝对值的某个比例。
4 把扰动加到权重上，得到"被扰动的模型"。
5 再前向-反向一次，用被扰动的模型计算损失与梯度，并据此更新原始权重（优化器 step）。
6 恢复被扰动前的权重副本（只保留更新带来的变化），继续下一个批次。

EMA

Top4



1. Data Preprocessing

- First, I used StratifiedKFold split the data into 5 folds. Then, I selected a fold with a cross-validation score that was slightly below average as the validation set, in order to avoid using folds with scores that were either too high or too low.
- As discussed here. In train data there are 12 wrong category labels. Namely 12 rows with QuestionId 31778 have MC_Answer 9 and Category True when the correct answer is 6. Therefore, I have applied the same processing method described in the discussion thread during both the training and inference stages.

2.Modeling

- Since the training set and the online test set share the same 15 questions, determining whether the selected answer is correct does not require prediction. Therefore, the task can be simplified to only predicting the type of explanation (Correct, Misconception, or Neither in Category), which reduces the number of prediction labels from 65 to 37.
- Based on the provided train.csv, it can be statistically determined that each question has between 4 to 6 candidate targets, which further reduces the target space.

```
train['candidate_targets'] = train['QuestionId'].map(train.groupby("QuestionId")
['target'].apply(set).apply(list))
train['neg_candidate_targets'] = train.apply(lambda x : [y for y in x["candidate_targets"] if
x["target"] != y], axis=1)
```

While the online test set might present different scenarios—such as potential new misconception Labels for each question—I have observed that samples labeled as "Correct" and "Neither" constitute the majority. Therefore, I intend for the model to focus primarily on the patterns observed in the currently available samples.

- My ranker models was trained with a pointwise approach, where it sees one label at a time in the context window. As discussed here. I configured the training process so that each step processes only one row_id sample. Since each row_id has 4 to 6 different candidate labels, all of these must be included in a single batch, so the actual batch size is 4 to 6.

3.Backbone

The experimental models I tested are primarily from the Qwen series. The experimental results for each model are summarized from multi-fold data:

```
Qwen3-32B >= Qwen3-Reranker-8B >= QWQ-32B >= Qwen3-14B >= Qwen2.5-Math-7B-Instruct > Qwen2.5-14B-Instruct >
Qwen3-8B > Qwen2.5-7B-Instruct
```

Since Qwen3-Reranker-8B and Qwen2.5-Math-7B-Instruct demonstrated outstanding performance, I decided to use Qwen3-Reranker-8B for iterative optimization, while employing the larger Qwen2.5-Math-72B-Instruct as the teacher model for knowledge distillation. Regarding the choice of the teacher model, due to my limited GPU resources, I did not experiment with many other large-scale models.

4.Effective Optimization Process

1. Example of Prompt_v1 (base prompt)

```
<|im_start|>system
You are an expert in detecting grade-school level math misconceptions. The task performs 2 steps:
1.Assesses whether the explanation contains a misconception. (Correct, Misconception, or Neither)
2.Identifies the specific misconception present, if any.<|im_end|>
<|im_start|>user
Question: What fraction of the shape is not shaded? Give your answer in its simplest form. [Image: A
triangle split into 9 equal smaller triangles. 6 of them are shaded.]
Student's Answer: \( \frac{1}{3} \)
This answer is correct.
Student's Explanation: I think that 1/3 is the answer, as it's the simplest form of 3/9.

Now we judge the student's explanation is Correct, Does this diagnosis is correct? (Yes/No)<|im_end|>
<|im_start|>assistant
```

- Prompt_v1 + Qwen3-Reranker-8B with 1 epoch gave me cv score is 0.9473, public score is 0.944, private score is 0.942
- Prompt_v1 + Qwen3-Reranker-8B + distill with 1 epoch gave me cv score is 0.9480, public score is 0.946, private score is 0.945

2.Example of prompt_2 (base prompt + few shots for target)

```
<|im_start|>system
You are an expert in detecting grade-school level math misconceptions. The task performs 2 steps:
1.Assesses whether the explanation contains a misconception. (Correct, Misconception, or Neither)
2.Identifies the specific misconception present, if any.<|im_end|>
<|im_start|>user
Question: What fraction of the shape is not shaded? Give your answer in its simplest form. [Image: A
triangle split into 9 equal smaller triangles. 6 of them are shaded.]
Student's Answer: \( \frac{1}{3} \)
This answer is correct.
Correct's Explanation Samples:
- Because there are 9 triangles and 3 arent shaded it is 3/9 and they are both multiples of 3 so it's
1/3
- The answer is 3/9 which can be simplified to 1/3
- I think this because there are 9 triangles and 3 are not shaded and 3/9 is 1/3.
- there are 9 triangles shaded and 3 are not shaded=3/9=1/3
- I think this because the answer is three over nine then they can be simplified to one over three.
- There are 9 triangles and 3 is not shaded. 3/9 is a possibility but the question says to put it in
its simplest form. 9 divided by 3 is 3 and 3 divided by 3 is one. You cannot simplify it anymore so
the answer is 1/3.
--
Neither's Explanation Samples:
- i counted the in-shaded parts then the shaded part then i made them a fraction.
- there are 9 triangles in total and 3 are not shaded. this means that 3 times 9 is 3 and 1 is 9.
- I think this because it is hard of the number
- 1/3. 3/9 shaded, and to find simplest from we divide the top and the bottom by 3.
- Because I counted them and seen the awnser
- I think this is because the fraction of the shape shaded is 3/9 but simplified it is 1/3.
--
Misconception:Incomplete's Explanation Samples:
- there are 9 boxes altogether and 3 of them are not shaded.
- there are 9 triangles andd 3 are not shaded
- Because there is nine triangular spaces and three are unshaded
- There are 9 squares but as you can see theree are 3 not shaded.
- I think this because there are nine triangles and thee are not shaded so it's three over nine.
- i think this because 3 out of nine triangles are not shaded
--
Misconception:WNB's Explanation Samples:
- 6 is the total and 3 is blank
- because 3 arr white and 6 are blue.
- 6 triangles are shaded so 3 aren't shaded
- 3 of the triangles are not shaded and 6 are shaded so therefore it is 3/6
- i counted the white ones and the blue ones.
- because 3 are shaded and 6 are not
--
Student's Explanation: I think that 1/3 is the answer, as it's the simplest form of 3/9.

Now we judge the student's explanation exhibited the misconception of WNB, Does this diagnosis is
correct? (Yes/No)<|im_end|>
<|im_start|>assistant
```

1).Prompt_v2 + Qwen3-Reranker-8B + distill with 1 epoch gave me cv score is 0.9493, public score is 0.948, private score is 0.946, Regarding the number of few-shot examples in prompt_v2, I randomly sampled 1 to 3 examples per target during training, and 6 examples per target during online inference.
2).Prompt_v2 + Qwen3-Reranker-8B + distill + full data with 1 epoch gave me public score is 0.950, private score is 0.947.
3).Prompt_2 + Qwen3-Reranker-8B + distill + full data with 2 epoch gave me private score is 0.951, public score is 0.948.(**best single model**), Here's an interesting episode: the model trained on the full dataset for 2 epochs encountered issues during online inference. Since the T4 GPU does not support bfloat16 type in vllm, converting the model to float16 resulted in overflow and produced a large number of NaN values. By this time, the competition was nearing its deadline, leaving me with limited time for adjustments. As an alternative, I used the 2-epoch model from the local validation phase as the backtone, augmented it with validation data, and trained for another 2 epochs to approximate the effect of training on the full dataset for 2 epochs.
4).prompt_v1 + Qwen3-32B + distill with 1 epoch gave me cv score is 0.9502, After using full data, public score is 0.949, private score is 0.946.
5).I performed an ensemble using Model 3 (my best single model) and Model 4. The ensemble achieved a public score of 0.952 and a private score of 0.948. This was one of my final selected submissions. However, since Model 4, despite having a high cv score and public score, demonstrated relatively lower performance on the private leaderboard, the final result of the ensemble did not surpass that of my best single model.

Submission and Description	Private Score	Public Score	Selected
th_eedi_v2_blend_model_v3 - Version 1	0.949	0.951	☐
th_eedi_v2_blend_model_v3 - Version 2	0.949	0.951	☐
th_eedi_v2_base_model_v4 - Version 9	0.948	0.951	☐
th_eedi_v2_blend_model_v2 - Version 4	0.948	0.951	☐
th_eedi_v2_blend_model_v9 - Version 2	0.948	0.951	☐
th_eedi_v2_blend_model_v2 - Version 8	0.948	0.950	☐
th_eedi_v2_blend_model_v9 - Version 1	0.948	0.952	☑
th_eedi_v2_blend_model_v2 - Version 2	0.948	0.951	☐
th_eedi_v2_blend_model_v2 - Version 5	0.948	0.951	☐
th_eedi_v2_blend_model_v2 - Version 3	0.948	0.951	☐
Fork of th_eedi_v2_blend_model_v3 - Version 1	0.948	0.951	☐

3.conclusion

model	task	cv	lb	pb	use full	inferrence time
prompt_v1 + Qwen3-Reranker-8B + 1 epoch	CausalLM	0.9473	0.944	0.942		40 min
prompt_v1 + Qwen3-Reranker-8B + distill + 1 epoch	CausalLM	0.9480	0.946	0.945		40 min
prompt_v2 + Qwen3-Reranker-8B + distill + 1 epoch	CausalLM	0.9493	0.948	0.946		4 hours
prompt_v2 + Qwen3-Reranker-8B + distill + 1 epoch	CausalLM	0.9493	0.950	0.947	✅	4 hours
prompt_v2 + Qwen3-Reranker-8B + distill + 2 epoch	CausalLM	0.9499	0.951	0.948	✅	4 hours
prompt_v1 + Qwen3-32B-AWQ + distill + 1 epoch	CausalLM	0.9502	0.949	0.946	✅	2.5 hours

5.Not Worked for me

- listwist ranker
- chain of thought: I personally spent the most time on Chain-of-Thought (COT), but it ultimately didn't yield any improvements.
- add the most relevant explanation to prompt: I fine-tuned a retrieval model (Qwen3-Emebedding-8B) to find the N most relevant explanations to the current student explanation and incorporated them into the prompt. This method achieved a decent CV score of 0.9493, but get worse score on both the public and private leaderboards.

```
<|im_start|>system
You are an expert in detecting grade-school level math misconceptions. The task performs 2 steps:
1.Assesses whether the explanation contains a misconception. (Correct, Misconception, or Neither)
2.Identifies the specific misconception present, if any.<|im_end|>
<|im_start|>user
Question: What fraction of the shape is not shaded? Give your answer in its simplest form. [Image: A
triangle split into 9 equal smaller triangles. 6 of them are shaded.]
Student's Answer: \( \frac{1}{3} \)
This answer is correct.
Top relevant explanations with their tags:
- Because there are 9 triangles and 3 of them were not shaded (Correct)
- because there is 9 triangles and 3 of them were not shaded (Correct)
- there aree 9 triangles in total and 3 of them are not shaded. this gives you 3 / 9. (Correct)
--
Student's Explanation: I think that 1/3 is the answer, as it's the simplest form of 3/9.

Now we judge the student's explanation is Correct, Does this diagnosis is correct? (Yes/No)<|im_end|>
<|im_start|>assistant
```

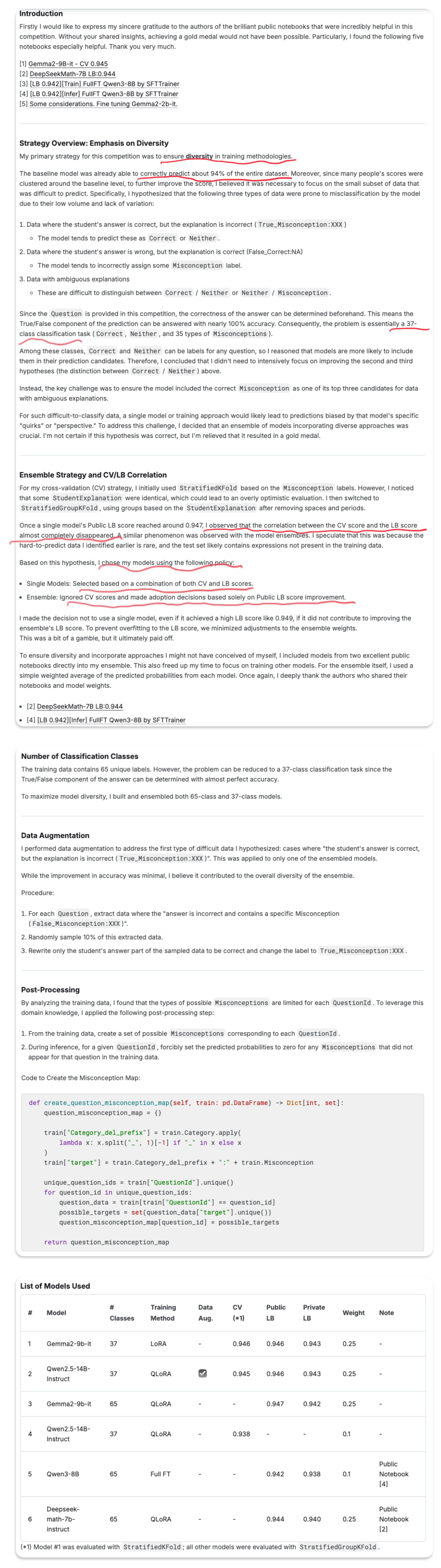
6.Not attempted

Generate more explanations for data augmentation

7.Reference

kaggle_eedi
offline-install-vllm-0.10.0
Qwen
vllm

Top5



Introduction

Firstly I would like to express my sincere gratitude to the authors of the brilliant public notebooks that were incredibly helpful in this competition. Without your shared insights, achieving a gold medal would not have been possible. Particularly, I found the following five notebooks especially helpful. Thank you very much.

[1] Gemma2-9b-it - CV 0.945
[2] DeepSeekMath-7B LB:0.944
[3] [LB 0.942][Train] FullFT Qwen3-8B by SFTTrainer
[4] [LB 0.942][Infer] FullFT Qwen3-8B by SFTTrainer
[5] Some considerations. Fine tuning Gemma2-2b-it.

Strategy Overview: Emphasis on Diversity

My primary strategy for this competition was to ensure diversity in training methodologies.

The baseline model was already able to correctly predict about 94% of the entire dataset. Moreover, since many people's scores were clustered around the baseline level, to further improve the score, I believed it was necessary to focus on the small subset of data that was difficult to predict. Specifically, I hypothesized that the following three types of data were prone to misclassification by the model due to their low volume and lack of variation:

1. Data where the student's answer is correct, but the explanation is incorrect (`True_Misconception:XXX`)
    - The model tends to predict these as `Correct` or `Neither`.
2. Data where the student's answer is wrong, but the explanation is correct (False_Correct:NA)
    - The model tends to incorrectly assign some `Misconception` label.
3. Data with ambiguous explanations
    - These are difficult to distinguish between `Correct` / `Neither` or `Neither` / `Misconception`.

Since the `Question` is provided in this competition, the correctness of the answer can be determined beforehand. This means the True/False component of the prediction can be answered with nearly 100% accuracy. Consequently, the problem is essentially a 37-class classification task (`Correct`, `Neither`, and 35 types of `Misconceptions`).

Among these classes, `Correct` and `Neither` can be labels for any question, so I reasoned that models are more likely to include them in their prediction candidates. Therefore, I concluded that I didn't need to intensively focus on improving the second and third hypotheses (the distinction between `Correct` / `Neither`) above.

Instead, the key challenge was to ensure the model included the correct `Misconception` as one of its top three candidates for data with ambiguous explanations.

For such difficult-to-classify data, a single model or training approach would likely lead to predictions biased by that model's specific "quirks" or "perspective." To address this challenge, I decided that an ensemble of models incorporating diverse approaches was crucial. I'm not certain if this hypothesis was correct, but I'm relieved that it resulted in a gold medal.

Ensemble Strategy and CV/LB Correlation

For my cross-validation (CV) strategy, I initially used `StratifiedKFold` based on the `Misconception` labels. However, I noticed that some `StudentExplanation` were identical, which could lead to an overly optimistic evaluation. I then switched to `StratifiedGroupKFold`, using groups based on the `StudentExplanation` after removing spaces and periods.

Once a single model's Public LB score reached around 0.947, I observed that the correlation between the CV score and the LB score almost completely disappeared. A similar phenomenon was observed with the model ensembles. I speculate that this was because the hard-to-predict data I identified earlier is rare, and the test set likely contains expressions not present in the training data.

Based on this hypothesis, I chose my models using the following policy:

- Single Models: Selected based on a combination of both CV and LB scores.
- Ensemble: Ignored CV scores and made adoption decisions based solely on Public LB score improvement.

I made the decision not to use a single model, even if it achieved a high LB score like 0.949, if it did not contribute to improving the ensemble's LB score. To prevent overfitting to the LB score, we minimized adjustments to the ensemble weights. This was a bit of a gamble, but it ultimately paid off.

To ensure diversity and incorporate approaches I hadn't conceived of myself, I included models from two excellent public notebooks directly into my ensemble. This also freed up my time to focus on training other models. For the ensemble itself, I used a simple weighted average of the predicted probabilities from each model. Once again, I deeply thank the authors who shared their notebooks and model weights.

- [2] DeepSeekMath-7B LB:0.944
- [4] [LB 0.942][Infer] FullFT Qwen3-8B by SFTTrainer

Number of Classification Classes

The training data contains 65 unique labels. However, the problem can be reduced to a 37-class classification task since the True/False component of the answer can be determined with almost perfect accuracy.

To maximize model diversity, I built and ensembled both 65-class and 37-class models.

Data Augmentation

I performed data augmentation to address the first type of difficult data I hypothesized: cases where "the student's answer is correct, but the explanation is incorrect (`True_Misconception:XXX`)". This was applied to only one of the ensembled models.

While the improvement in accuracy was minimal, I believe it contributed to the overall diversity of the ensemble.

Procedure:

1. For each `Question`, extract data where the "answer is incorrect and contains a specific Misconception (`False_Misconception:XXX`)".
2. Randomly sample 10% of this extracted data.
3. Rewrite only the student's answer part of the sampled data to be correct and change the label to `True_Misconception:XXX`.

Post-Processing

By analyzing the training data, I found that the types of possible `Misconceptions` are limited for each `QuestionId`. To leverage this domain knowledge, I applied the following post-processing step:

1. From the training data, create a set of possible `Misconceptions` corresponding to each `QuestionId`.
2. During inference, for a given `QuestionId`, forcibly set the predicted probabilities to zero for any `Misconceptions` that did not appear for that question in the training data.

Code to Create the Misconception Map:

```
def create_question_misconception_map(self, train: pd.DataFrame) -> Dict[int, set]:
    question_misconception_map = {}

    train["Category_del_prefix"] = train.Category.apply(
        lambda x: x.split("_", 1)[-1] if "_" in x else x
    )
    train["target"] = train.Category_del_prefix + ":" + train.Misconception

    unique_question_ids = train["QuestionId"].unique()
    for question_id in unique_question_ids:
        question_data = train[train["QuestionId"] == question_id]
        possible_targets = set(question_data["target"].unique())
        question_misconception_map[question_id] = possible_targets

    return question_misconception_map
```

List of Models Used

#	Model	# Classes	Training Method	Data Aug.	CV (*1)	Public LB	Private LB	Weight	Note
1	Gemma2-9b-it	37	LoRA	-	0.946	0.946	0.943	0.25	-
2	Qwen2.5-14B-Instruct	37	QLoRA	☑	0.945	0.946	0.943	0.25	-
3	Gemma2-9b-it	65	QLoRA	-	-	0.947	0.942	0.25	-
4	Qwen2.5-14B-Instruct	37	QLoRA	-	0.938	-	-	0.1	-
5	Qwen3-8B	65	Full FT	-	-	0.942	0.938	0.1	Public Notebook [4]
6	Deepseek-math-7b-instruct	65	QLoRA	-	-	0.944	0.940	0.25	Public Notebook [2]

(*1) Model #1 was evaluated with `StratifiedKFold`; all other models were evaluated with `StratifiedGroupKFold`.

Prompts Used

I used different prompts for each model. (Variable omitted.)

Model # 1

```
<start_of_turn>user
You are an expert math educator analyzing student responses for mathematical misconceptions.

### TASK
Your goal is to analyze the student's explanation for a given math problem and classify it into a
specific category of understanding or misconception.

### INPUT
- Question:{row['QuestionText']}
- Answer Choices:{choices}
- Student's Answer:{row['MC_Answer']}
- Correct?:{correctness}
- Student's Explanation:{row['StudentExplanation']}
- Classification:<end_of_turn>
<start_of_turn>model
```

Model #2

```
<|im_start|>user
Question: {row["QuestionText"]}
Answer: {row["MC_Answer"]}
{correctness}
Student Explanation: {row["StudentExplanation"]}<|im_end|>
<|im_start|>assistant
```

Model #3

```
<start_of_turn>user
You are an expert math educator analyzing student responses for mathematical misconceptions.

Question: {question}
Correct Answer: {answer}
Student's Explanation: {explanation}

CLASSIFICATION GUIDELINES:
• True_Correct:NA = Student demonstrates correct understanding
• False_Correct:NA = Student gives correct answer but for wrong reasons
• True_Neither:NA = Correct answer but unclear/incomplete reasoning
• False_Neither:NA = Incorrect answer but no specific misconception identified
• True_Misconception:[Type] = Correct answer but demonstrates specific misconception
• False_Misconception:[Type] = Incorrect answer with identifiable misconception

TASK: Classify this student's response using EXACTLY ONE of these {len(all_labels)} labels:

{labels_text}

Classification:<end_of_turn>
<start_of_turn>model
```

Model #4

```
<|im_start|>user
Question: {row["QuestionText"]}
Ground Truth: {ground_truth}
Student Explanation: {row["StudentExplanation"]}<|im_end|>
<|im_start|>assistant
```